Lyric Match – Similarity & Ambient Detection Engine

(Lyrics Similarity Detection using KMP, Cosine and Jaccard Similarity with FFT-based Audio Fingerprinting)

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
Prajwal (2520080017)
Kavi Krishna (2520080021)
Hemanth (2520090074)

Team Number:

Section: 13

Under the guidance of

Dr. Dyuti Banerjee

Professor, Department of Artificial Intelligence and Data Science



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE



DECLARATION

The Mini Project Report entitled "**Lyric Match – Similarity & Ambient Detection Engine"** is a record of Bonafide work of **Prajwal – 2520080017, Kavi Krishna – 2520080021, Hemanth – 2520090074** submitted in partial fulfillment for the award of B. Tech in Artificial Intelligence & Data Science to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Prajwal – 2520080017

Kavi Krishna – 2520080021

Hemanth – 2520090074

K L (Deemed to be) University

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE



CERTIFICATE

This is to certify that the mini project based report entitled "**Lyric Match – Similarity & Ambient Detection Engine"** is a bonafide work done and submitted by **Prajwal – 2520080017, Kavi Krishna – 2520080021, Hemanth – 2520090074** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Artificial Intelligence & Data Science, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to our guide, Dr. Dyuti Banerjee, Professor, Department of Artificial Intelligence and Data Science, which cannot be expressed in words for the tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Sreenath Itikela**, Head of the *Department of Artificial Intelligence and Data Science* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Prajwal – 2520080017

Kavi Krishna – 2520080021

Hemanth – 2520090074

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	9
5.	Hardware and Software requirements	10
6.	Implementation	10
7.	Results & Discussion	16
8.	Conclusion & Future Scope	19
9.	References	20

1.ABSTRACT

Lyric Match is a web-based detection engine that identifies lyrical similarity and recognises songs from ambient audio. The text module compares submitted lyrics against a multilingual reference catalogue of English, Hindi and Telugu tracks using three classical algorithms: the Knuth–Morris–Pratt (KMP) algorithm to locate exact copied phrases of three or more words, Cosine Similarity to compare word-frequency vectors, and Jaccard Similarity to compare vocabulary overlap. The final score is the mean of the cosine and Jaccard values, and a score of 45% or more flags the lyrics as similar. The audio module follows the fingerprinting approach used by commercial music-recognition services: a recording is split into Hann-windowed frames, converted to the frequency domain with a radix-2 Cooley–Tukey Fast Fourier Transform, reduced to spectral peaks in six frequency bands, and turned into hashes of peak pairs. These hashes are looked up in an index built from the reference tracks, and the song is identified by an offset-histogram vote. On a sample query that paraphrases the song “Neon Skyline”, the system ranked it first with a combined score of 75.7% (cosine 86.8%, Jaccard 64.6%) and highlighted five copied phrases covering 33 words. The whole system runs client-side in the browser using HTML, CSS and JavaScript, so no lyrics or audio leave the user’s device.

Team Name:

Team Logo (if any):

Team Members:

1. Prajwal, 2520080017
2. Kavi Krishna, 2520080021
3. Hemanth, 2520090074

2. INTRODUCTION

Music catalogues now contain millions of songs, and new lyrics are written every day. Two practical questions follow from this scale. First, how can we tell whether a new set of lyrics copies, or is very close to, an existing song? Second, how can we find out which song is playing when we only hear a few seconds of it in the background? Doing either by hand is impractical, which makes both problems good candidates for algorithmic solutions.

Lyric Match addresses both questions in a single single-page web application. For text, it compares a query against a catalogue of reference lyrics and reports a ranked similarity score together with the evidence behind it: a side-by-side view with copied phrases highlighted. For audio, it listens through the microphone or reads an uploaded clip, converts the sound into a compact fingerprint and matches it against fingerprints of the reference tracks. The tokeniser is Unicode-aware, so English, Hindi and Telugu lyrics are handled by the same code.

The project is built around topics from Data Structures and Algorithms: string matching with the KMP failure function, hash sets and hash maps for set and frequency operations, divide-and-conquer in the Fast Fourier Transform, hashing for the fingerprint index, and sorting for ranking. Each of these algorithms has a clear role in the system and a known time complexity, which is analysed in the Results section.

The rest of this report is organised as follows. Section 3 reviews related systems and algorithms. Section 4 states the problem and objectives. Section 5 lists the hardware and software requirements. Section 6 presents the implementation of the four main modules. Section 7 reports and discusses the results. Section 8 concludes the report and outlines future scope, and Section 9 lists the references.

3. LITERATURE SURVEY

Shazam and the landmark fingerprinting algorithm [1][2]. Shazam identifies a song from a few seconds of noisy audio. Avery Wang's 2003 paper describes the core idea: find peaks in the spectrogram, pair nearby peaks into hashes, and confirm a match by checking that many hashes line up at the same time offset. The method tolerates noise and compression. Lyric Match uses a simplified version of this approach for its audio module.

Dejavu [3]. Dejavu is an open-source Python library that implements audio fingerprinting in the same spirit as Shazam and stores the fingerprints in a relational database. It can recognise songs from files and from microphone input. Lyric Match differs in that the index is built in memory inside the browser, so it needs no server or database.

Chromaprint and AcoustID [4]. Chromaprint computes a fingerprint from chroma features, which describe how the energy of a sound is spread over musical pitch classes, and AcoustID uses it to identify recordings for music tagging. It focuses on matching whole recordings rather than short ambient snippets. Our system instead uses spectral peak pairs, which are better suited to short and noisy captures.

MOSS (Measure Of Software Similarity) [5]. MOSS, developed at Stanford, detects similarity between programs by selecting fingerprints from the text and comparing them across submissions. It shows that fingerprinting and matching can reveal copying even when small edits are made. Lyric Match applies the same idea to lyrics, and also shows the exact matching passages as evidence.

Turnitin [6]. Turnitin is a commercial service that checks written work against a very large database and reports a similarity percentage with the matching passages highlighted. Its report format influenced our results screen, which also gives an overall percentage and highlighted evidence. Unlike Turnitin, Lyric Match is specialised for lyrics and works across English, Hindi and Telugu.

Knuth–Morris–Pratt string matching [7]. KMP finds a pattern in a text in linear time by precomputing a failure (longest-proper-prefix-suffix) table, so that characters are never compared twice after a mismatch. It is the classical answer to naive string search, which can take quadratic

time. Lyric Match uses KMP to locate copied phrases of three to eight words inside each reference song.

Vector space model and cosine similarity [8]. Salton, Wong and Yang proposed representing documents as vectors of term weights and measuring their similarity by the cosine of the angle between them. Cosine similarity rewards documents that use the same words in similar proportions and is not affected by document length. We use raw word-frequency vectors for the cosine measure.

Jaccard similarity [9]. The Jaccard index, introduced by Paul Jaccard in 1901, is the size of the intersection of two sets divided by the size of their union. It ignores how often a word is repeated and measures only vocabulary overlap. Combined with cosine similarity, it balances frequency-based and set-based views of similarity.

Fast Fourier Transform [10]. The Cooley–Tukey algorithm computes the discrete Fourier transform in O(N log N) time instead of $O(N^2)$ by recursively splitting the input. It is what makes frame-by-frame spectral analysis practical in real time. Our audio module uses an iterative radix-2 version of it.

Summary of the gap: existing tools are either audio-only recognisers or document-level plagiarism checkers, and few handle Indian-language lyrics. Lyric Match combines an explainable text-similarity engine with an audio-fingerprinting engine in one lightweight, client-side application.

4. Problem Statement and Objectives

Problem Statement:

To build a client-side system that detects similarity between submitted lyrics and a catalogue of songs, and identifies a song from ambient audio, using string-matching, vector-similarity and audio-fingerprinting algorithms.

Problem Description:

Lyrics can be copied wholly or in part and in several languages, and a song heard in the background cannot be searched by name. A fast and explainable detector is needed that shows not only a score but also the evidence behind it.

Objectives:

- To implement the KMP algorithm to find exact copied phrases of three or more words between a query and each reference song.
- To compute Cosine and Jaccard similarity on Unicode-aware tokens (English, Hindi and Telugu) and combine them into a single ranked score.
- To present explainable results: a ranked table, side-by-side highlighted evidence and a list of matched phrases.
- To implement FFT-based audio fingerprinting with hashing and offset voting to identify songs from microphone input or an uploaded clip.
- To deliver the whole system as a responsive single-page web application that runs entirely in the browser.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Computer or laptop with a dual-core processor and at least 4 GB RAM (8 GB recommended)
- Built-in or external microphone (needed for ambient detection)
- Speakers or headphones for playing reference tracks during testing
- Display of 1366 × 768 or higher (the layout is also responsive for phones)

Software Requirements:

- Operating system: Windows 10/11, Linux or macOS
- Modern web browser (Google Chrome, Microsoft Edge or Firefox) with Web Audio API and microphone support
- HTML5, CSS3 and JavaScript (ES6) – no external libraries and no server required
- Visual Studio Code (or any text editor) for development
- Unicode fonts with Devanagari and Telugu support for displaying Hindi and Telugu lyrics

6. IMPLEMENTATION

The system is organised into four modules. The first two form the text similarity engine and the last two form the ambient audio detection engine. The source code of each module is given below (taken from the project's JavaScript file).

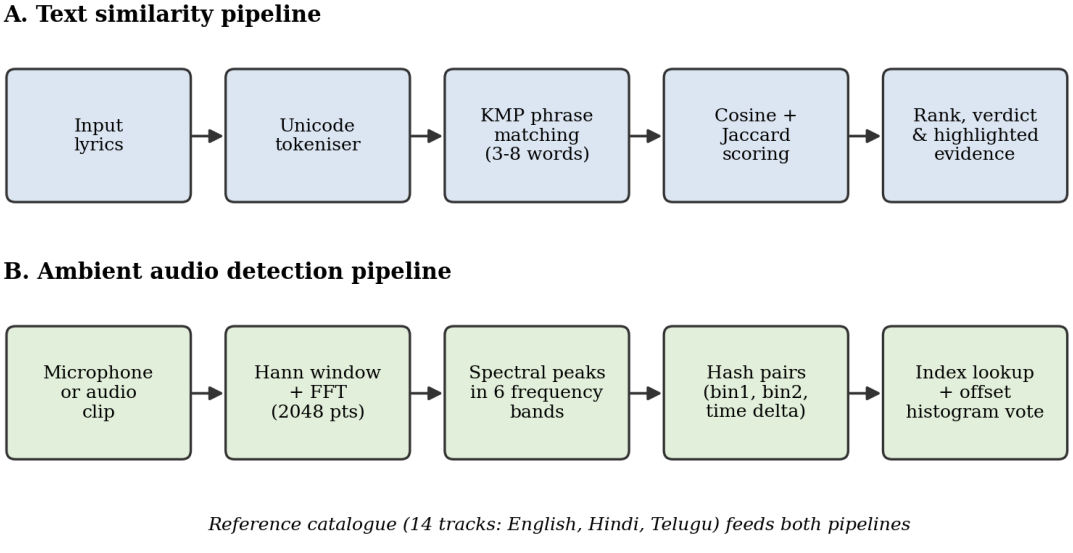


Fig. 6.1 Processing pipelines of the text and audio modules

Module 1 – Tokenisation and KMP phrase matching

The tokeniser lower-cases the text and extracts Unicode letters, marks and digits, so it works for English, Hindi and Telugu. buildLPS() constructs the KMP failure table and kmpIndexOf() searches a phrase in linear time. findMatchedSegments() slides windows of 8 down to 3 words over the query and uses KMP to check whether each window occurs in the reference song; words already matched are not reused.

```
function tokenize(str) {
  return str.toLowerCase().match(/[\p{L}\p{M}\p{N}']+/gu) || [];
}

function buildLPS(pattern) {
  var lps = new Array(pattern.length).fill(0),
    len = 0,
    i = 1;
  while (i < pattern.length) {
    if (pattern[i] === pattern[len]) {
      len++;
      lps[i] = len;
      i++;
    } else if (len > 0) {
      len = lps[len - 1];
    } else {
      lps[i] = 0;
      i++;
    }
  }
  return lps;
}
function kmpIndexOf(text, pattern) {
  if (!pattern.length) return -1;
  var lps = buildLPS(pattern),
    i = 0,
    j = 0;
  while (i < text.length) {
    if (text[i] === pattern[j]) {
```

```
      i++;
      j++;
      if (j === pattern.length) return i - j;
    } else if (j > 0) {
      j = lps[j - 1];
    } else {
      i++;
    }
  }
  return -1;
}
function findMatchedSegments(queryTokens, refCleanText) {
  var used = new Array(queryTokens.length).fill(false),
    found = [];
  var maxSize = Math.min(8, queryTokens.length);
  for (var size = maxSize; size >= 3; size--) {
    for (var i = 0; i + size <= queryTokens.length; i++) {
      var overlap = false;
      for (var k = i; k < i + size; k++) {
        if (used[k]) {
          overlap = true;
          break;
        }
      }
      if (overlap) continue;
      var phrase = queryTokens.slice(i, i + size).join(" ");
      if (kmpIndexOf(refCleanText, phrase) !== -1) {
        found.push({ phrase: phrase, size: size, start: i });
        for (var k2 = i; k2 < i + size; k2++) used[k2] = true;
      }
    }
  }
  found.sort(function (a, b) {
    return a.start - b.start;
  });
  return found;
}
```

Module 2 – Cosine similarity, Jaccard similarity and scoring

Cosine similarity builds word-frequency maps for both texts and divides the dot product by the product of the vector lengths. Jaccard similarity uses two sets and divides the size of the intersection by the size of the union. The combined score is their average, and a score of 45% or more is reported as “Similarity detected”.

```
function freqMap(tokens) {
  var m = {};
  tokens.forEach(function (t) {
    m[t] = (m[t] || 0) + 1;
  });
  return m;
}
function cosineSimilarity(a0, b0) {
  var a = freqMap(a0),
    b = freqMap(b0);
  var vocab = new Set(Object.keys(a).concat(Object.keys(b)));
  var dot = 0,
    na = 0,
    nb = 0;
  vocab.forEach(function (w) {
    var av = a[w] || 0,
      bv = b[w] || 0;
    dot += av * bv;
    na += av * av;
    nb += bv * bv;
  });
  if (na === 0 || nb === 0) return 0;
  return dot / (Math.sqrt(na) * Math.sqrt(nb));
```

```
}
function jaccardSimilarity(a0, b0) {
  var a = new Set(a0),
    b = new Set(b0),
    inter = 0;
  a.forEach(function (w) {
    if (b.has(w)) inter++;
  });
  var union = new Set(Array.from(a).concat(Array.from(b))).size;
  return union === 0 ? 0 : inter / union;
}

// combined score and decision (inside the Run handler)
var cosine = cosineSimilarity(queryTokens, refTokens);
var jaccard = jaccardSimilarity(queryTokens, refTokens);
var score = (cosine + jaccard) / 2;
var flagged = Math.round(score * 100) >= 45; // similarity threshold
```

Module 3 – FFT and spectral peak extraction

The audio is cut into 2048-sample frames with a hop of 512 samples. Each frame is multiplied by a Hann window and transformed with an iterative radix-2 Cooley–Tukey FFT. In each of six frequency bands the bin with the highest magnitude is kept as a peak, and very quiet frames are skipped.

```
// ---- FFT: iterative radix-2 Cooley-Tukey (N must be a power of 2) ----
function fft(re, im) {
  var n = re.length;
  for (var i = 1, j = 0; i < n; i++) {
    var bit = n >> 1;
    for (; j & bit; bit >>= 1) j ^= bit;
    j ^= bit;
    if (i < j) {
      var tr = re[i];
      re[i] = re[j];
      re[j] = tr;
      var ti = im[i];
      im[i] = im[j];
      im[j] = ti;
    }
  }
  for (var len = 2; len <= n; len <<= 1) {
    var ang = (-2 * Math.PI) / len,
      wr = Math.cos(ang),
      wi = Math.sin(ang);
    for (var i2 = 0; i2 < n; i2 += len) {
      var curWr = 1,
        curWi = 0;
      for (var j2 = 0; j2 < len / 2; j2++) {
        var ur = re[i2 + j2],
          ui = im[i2 + j2];
        var vr = re[i2 + j2 + len / 2] * curWr - im[i2 + j2 + len / 2] * curWi;
        var vi = re[i2 + j2 + len / 2] * curWi + im[i2 + j2 + len / 2] * curWr;
        re[i2 + j2] = ur + vr;
        im[i2 + j2] = ui + vi;
        re[i2 + j2 + len / 2] = ur - vr;
        im[i2 + j2 + len / 2] = ui - vi;
        var nwr = curWr * wr - curWi * wi,
          nwi = curWr * wi + curWi * wr;
        curWr = nwr;
        curWi = nwi;
      }
    }
  }
}

var WIN = 2048,
```

```
  HOP = 512;
var HANN = new Float64Array(WIN);
for (var hi = 0; hi < WIN; hi++)
  HANN[hi] = 0.5 - 0.5 * Math.cos((2 * Math.PI * hi) / (WIN - 1));
var BANDS = [
  [2, 10],
  [10, 20],
  [20, 40],
  [40, 80],
  [80, 160],
  [160, 232],
]; // FFT bin ranges (~ up to ~5kHz at 44.1kHz)

function extractPeaks(samples, sampleRate) {
  var peaks = [],
    frame = 0;
  var reBuf = new Float64Array(WIN),
    imBuf = new Float64Array(WIN);
  for (var pos = 0; pos + WIN <= samples.length; pos += HOP, frame++) {
    var rms = 0;
    for (var i = 0; i < WIN; i++) {
      var s = samples[pos + i] * HANN[i];
      reBuf[i] = s;
      imBuf[i] = 0;
      rms += s * s;
    }
    rms = Math.sqrt(rms / WIN);
    if (rms < 0.0006) continue;
    fft(reBuf, imBuf);
    for (var b = 0; b < BANDS.length; b++) {
      var lo = BANDS[b][0],
        hiB = BANDS[b][1],
        bestBin = -1,
        bestMag = -1;
      for (var bin = lo; bin < hiB; bin++) {
        var mag = reBuf[bin] * reBuf[bin] + imBuf[bin] * imBuf[bin];
        if (mag > bestMag) {
          bestMag = mag;
          bestBin = bin;
        }
      }
      if (bestBin >= 0 && bestMag > 8e-7) peaks.push({ t: frame, bin: bestBin });
    }
  }
  return peaks;
}
```

Module 4 – Fingerprint hashing and offset-histogram matching

Each peak (the anchor) is paired with up to five later peaks within 40 frames, and each pair becomes a hash of the two frequency bins and the time difference. A captured recording is hashed in the same way; every hash found in the reference index votes for a song and a time offset, and the song whose most popular offset has the most votes is reported as the match.

```
function buildHashes(peaks) {
  peaks.sort(function (a, b) {
    return a.t - b.t;
  });
  var hashes = [];
  for (var i = 0; i < peaks.length; i++) {
    var anchor = peaks[i],
      count = 0;
    for (var j = i + 1; j < peaks.length && count < 5; j++) {
      var target = peaks[j],
        dt = target.t - anchor.t;
      if (dt <= 0) continue;
      if (dt > 40) break;
```

```
      var b1 = anchor.bin,
        b2 = target.bin;
      hashes.push({ key: b1 + "_" + b2 + "_" + dt, t: anchor.t });
      count++;
    }
  }
  return hashes;
}

function matchBuffer(samples, sampleRate) {
  var peaks = extractPeaks(samples, sampleRate);
  var hashes = buildHashes(peaks);
  if (!hashes.length) return { rows: [], hashCount: 0 };
  var offsetHist = {};
  hashes.forEach(function (h) {
    var matches = refIndex[h.key];
    if (!matches) return;
    matches.forEach(function (m) {
      var offset = m.t - h.t;
      offsetHist[m.id] = offsetHist[m.id] || {};
      offsetHist[m.id][offset] = (offsetHist[m.id][offset] || 0) + 1;
    });
  });
  var rows = AUDIO_REFS.map(function (song) {
    var hist = offsetHist[song.id],
      best = 0;
    if (hist)
      Object.keys(hist).forEach(function (off) {
        best = Math.max(best, hist[off]);
      });
    return { song: song, raw: best, score: hashes.length ? best / hashes.length : 0 };
  }).sort(function (a, b) {
    return b.raw - a.raw;
  });
  return { rows: rows, hashCount: hashes.length };
}
```

7. RESULTS & DISCUSSION

7.1 Text similarity module

The text module was tested with a sample query that is a lightly reworded copy of the reference song “Neon Skyline” (46 tokens). It was compared with all 14 tracks of the reference catalogue (6 English, 4 Hindi and 4 Telugu). Table 7.1 shows the top six results.

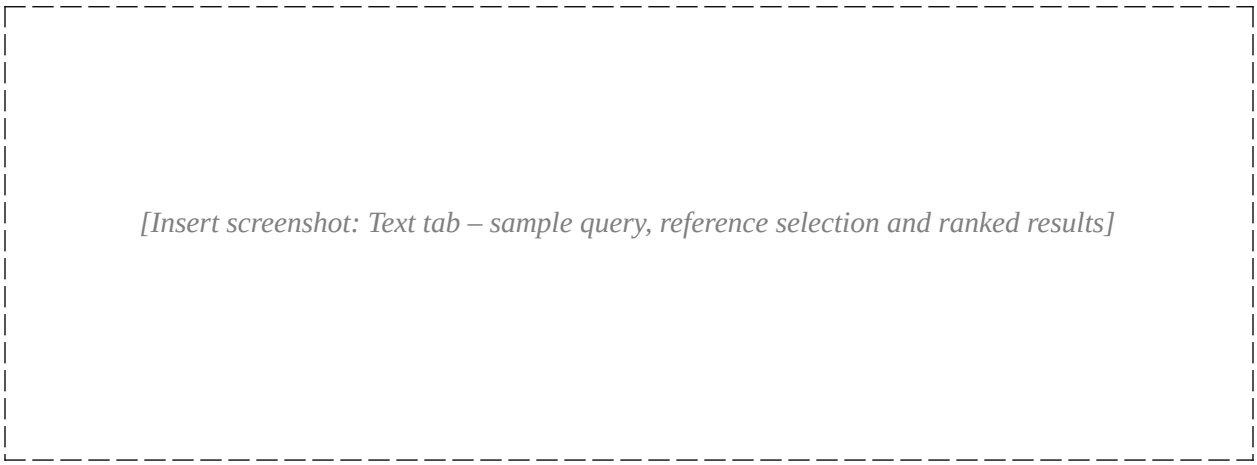


Fig. 7.1 Text similarity screen with the ranked results and the verdict banner

Table 7.1 Similarity scores for the sample query (46 tokens)

Rank	Reference track	Cosine	Jaccard	Combined	Matched words
1	Neon Skyline	86.8%	64.6%	75.7%	33
2	Paper Boats	48.3%	8.8%	28.5%	0
3	Golden Hour Girl	44.3%	9.5%	26.9%	0
4	Static on the Radio	39.4%	11.8%	25.6%	0
5	Midnight Train	38.3%	10.3%	24.3%	0
6	Embers & Ash	33.1%	11.3%	22.2%	0

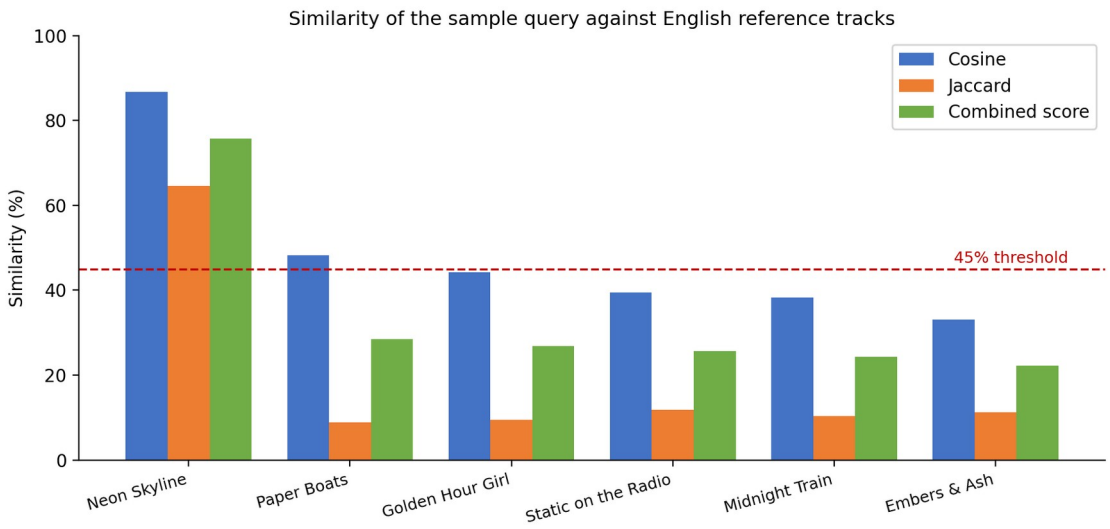


Fig. 7.2 Cosine, Jaccard and combined scores for the top six tracks

Table 7.2 Phrases of three or more words found by KMP in "Neon Skyline"

No.	Copied phrase found by KMP	Words
1	running through the neon skyline chasing headlights down	8
2	street your hand in mine	5
3	the traffic light the city hums a	7
4	we cannot beat every window	5
5	a story we will never know but tonight	8

The top result, "Neon Skyline", scored 75.7% (cosine 86.8%, Jaccard 64.6%), which is well above the 45% threshold, so the query is flagged as "Similarity detected". KMP found five copied phrases covering 33 of the 46 query words (Table 7.2), which shows that the high score comes from real copied wording and not just shared vocabulary.

Cosine similarity is higher than Jaccard similarity for the matching song because words such as "the" and "we" are repeated, and frequency-based cosine rewards that repetition, while Jaccard counts each distinct word once. The other English tracks score between 22% and 29%, only because they share common words, and none of them has a single copied phrase of three words or more. This is the reason for combining the two scores with KMP evidence.

All eight Hindi and Telugu tracks scored 0%. The query is in English and the tokens of different scripts never overlap, so the comparison is meaningful only within the same language. A Hindi or Telugu query is matched against Hindi or Telugu references in the same way.

7.2 Ambient audio detection module

The ambient detection tab builds the fingerprint index of the reference tracks when the page loads. The user can tap once to record for six seconds, switch on continuous listening, or upload a clip. The result shows the closest track, the confidence (aligned hashes divided by total hashes), and a sensitivity slider that sets the confidence needed to declare a match. Detections above the threshold are added to a history log.

[Insert screenshot: Ambient tab – Tap to identify, listening state and detection result]

Fig. 7.3 Ambient detection screen showing the closest fingerprint match and confidence

7.3 Complexity summary

Table 7.3 Algorithms used and their time complexity

Algorithm / step	Used for	Time complexity
KMP search (one phrase)	Exact phrase matching	O(L + m) for reference length L and phrase length m
Window search over query	Finding copied segments	At most 6 window sizes × q start positions KMP searches
Cosine similarity	Frequency-vector similarity	O(q + L) using hash maps
Jaccard similarity	Vocabulary overlap	O(q + L) using hash sets
FFT (per frame, N = 2048)	Time to frequency conversion	O(N log N)
Hash generation	Fingerprint creation	O(5P) for P peaks (5 pairs per anchor)
Index lookup and voting	Song identification	O(1) average per hash (hash map)
Ranking	Ordering results	O(k log k) for k tracks (sorting)

8. CONCLUSION & FUTURE SCOPE

Conclusion

The project set out to detect lyrical similarity and to identify songs from ambient audio, and both goals were met. The text module combines KMP phrase matching with cosine and Jaccard similarity, supports English, Hindi and Telugu, and explains every result with highlighted evidence. On the sample query it ranked the correct reference song first with a combined score of 75.7% and found five copied phrases. The audio module implements the full fingerprinting pipeline of FFT, peak picking, hashing and offset voting, and runs in the browser without any server.

The project also gave practical experience with the Data Structures and Algorithms topics of the course: string matching, hashing, hash maps and sets, sorting and the divide-and-conquer FFT.

Limitations: the reference catalogue consists of 14 original sample lyrics and the audio references are short synthesised melodies, not recordings of real songs, so the results show that the method works but are not a measure of accuracy on a real music library.

Future Scope

Near term: enlarge the reference catalogue; remove common stop-words or use TF-IDF weighting so that frequent words matter less; make the 45% text threshold adjustable like the audio sensitivity slider.

Medium term: store fingerprints and lyrics in a database behind a small backend so that real songs can be indexed; support uploading real audio files as references; use n-gram shingling or winnowing to scale the text search to large catalogues.

Long term: add cross-language and meaning-based similarity using word embeddings; offer an API or browser extension; build a mobile app with real-time streaming recognition; export a similarity report as a PDF.

9. REFERENCES

[1] A. Wang, “An Industrial-Strength Audio Search Algorithm,” Proc. 4th International Conference on Music Information Retrieval (ISMIR), 2003. https://www.ee.columbia.edu/~dpwe/papers/Wang03-shazam.pdf

[2] Shazam music recognition service. https://www.shazam.com

[3] Dejavu – open-source audio fingerprinting in Python. https://github.com/worldveil/dejavu

[4] Chromaprint and AcoustID audio fingerprinting. https://acoustid.org/chromaprint

[5] MOSS – A System for Detecting Software Similarity, Stanford University. https://theory.stanford.edu/~aiken/moss/

[6] Turnitin similarity checking service. https://www.turnitin.com

[7] D. E. Knuth, J. H. Morris and V. R. Pratt, “Fast Pattern Matching in Strings,” SIAM Journal on Computing, vol. 6, no. 2, pp. 323–350, 1977. https://doi.org/10.1137/0206024

[8] G. Salton, A. Wong and C. S. Yang, “A Vector Space Model for Automatic Indexing,” Communications of the ACM, vol. 18, no. 11, pp. 613–620, 1975. https://doi.org/10.1145/361219.361220

[9] P. Jaccard, “Étude comparative de la distribution florale dans une portion des Alpes et des Jura,” Bulletin de la Société Vaudoise des Sciences Naturelles, vol. 37, pp. 547–579, 1901.

[10] J. W. Cooley and J. W. Tukey, “An Algorithm for the Machine Calculation of Complex Fourier Series,” Mathematics of Computation, vol. 19, no. 90, pp. 297–301, 1965. https://doi.org/10.1090/S0025-5718-1965-0178586-1